DevOps Project

Production-Grade Cloud-Native Application Deployment on AWS EKS

Architecture & Code Review

https://github.com/fasih6/End_to_End_Gitlab_CI-CD_on_AWS_EKS_Devops_Project.git

---

Project Stack

Layer	Technology
CI/CD	GitLab CI/CD (self-hosted runner)
Container Registry	Docker Hub
Cloud Provider	AWS (us-east-1)
Container Orchestration	EKS (Kubernetes 1.33)
Infrastructure as Code	Terraform >= 1.0
Observability	Prometheus · Grafana · Loki · Alertmanager
Ingress / TLS	NGINX Ingress + cert-manager (Let's Encrypt)
Database	MySQL 8 (StatefulSet + EBS PVC)
Code Quality	SonarQube
Security Scanning	Trivy + Gitleaks

1. Overall Assessment

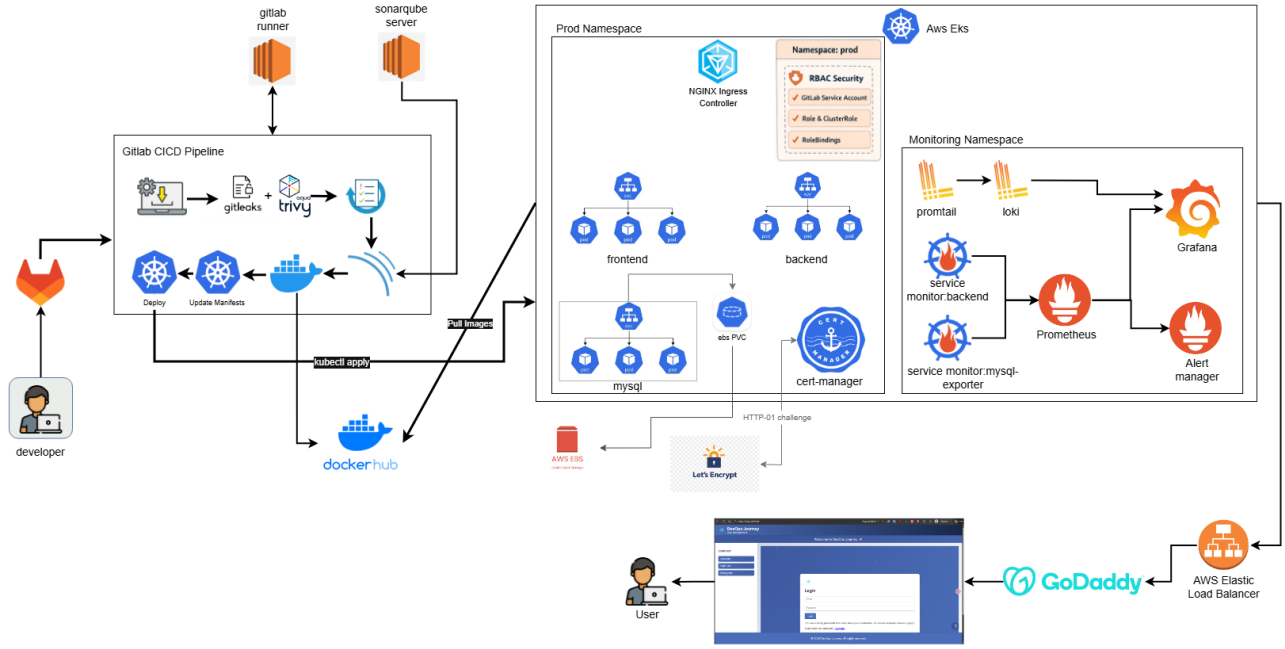
This is a well-architected, production-grade DevOps project demonstrating strong command of modern cloud-native practices. The pipeline design, Kubernetes resource definitions, Terraform infrastructure, and observability stack are all at a professional standard. The primary area requiring attention is credential hygiene — several secrets and default passwords are committed to the repository.

Component	Score	Status	Summary
GitLab CI/CD Pipeline	**9/10**	■ EXCELLENT	Well-structured, parallel scans, GitOps manifest update, SHA image tags
Kubernetes Manifests	**8.5/10**	■ EXCELLENT	RollingUpdate zero-downtime, named ports, resource limits, Prom annotations
Terraform / EKS	**9/10**	■ EXCELLENT	OIDC/IRSA, EBS CSI addon, IAM least-privilege, subnet tagging for LB
RBAC / Authentication	**8/10**	■ GOOD	Namespace Role + ClusterRole correctly separated; Token Secret correct
Observability Stack	**9/10**	■ EXCELLENT	Full kube-prometheus-stack, Loki+Promtail, ServiceMonitors, dashboards
Security Posture	**6/10**	■■ REVIEW	Hardcoded passwords in Git; /metrics exposed; SSH open to 0.0.0.0/0

2. Architecture Overview

The project follows a GitOps-oriented, cloud-native architecture. Source code changes trigger a multi-stage GitLab CI/CD pipeline that builds and secures container images before updating Kubernetes manifests and deploying to AWS EKS. All application state persists on AWS EBS volumes and the full observability stack runs inside the cluster.

2.1 High-Level Architecture Flow



2.2 Architecture Principles

- **GitOps:** Kubernetes manifests are the source of truth. The pipeline commits image tag changes back to Git before deploying, creating a full audit trail.
- **Immutable images:** Every commit produces a uniquely SHA-tagged Docker image. The latest tag is also updated but the manifest always references the SHA tag.
- **Least privilege:** RBAC is scoped to the prod namespace (Role/RoleBinding) with a minimal ClusterRole for storage. EBS CSI uses IRSA, not node instance profiles.
- **Zero-downtime deployments:** RollingUpdate with maxUnavailable: 0 ensures users are never served by fewer than the desired replica count.
- **Automated TLS:** cert-manager provisions and renews Let's Encrypt certificates automatically with no manual intervention.
- **Full observability:** Every service exposes Prometheus metrics. Logs aggregated via Loki/Promtail. Alerts routed to Slack/email via Alertmanager.

3. GitLab CI/CD Pipeline

The pipeline is composed of 7 sequential stages. Security scan jobs run in parallel within the security_scan stage to reduce overall pipeline duration.

3.1 Pipeline Stages

#	Stage	Job Name	Purpose	Runs On	Output
1	**install_tools**	install-tools-job	Installs Trivy + Gitleaks on self-hosted runner. Cached for reuse.	Always	—
2	**security_scan**	gitleaks-scan-job	Scans full Git history for leaked secrets, API keys, credentials (Gitleaks).	Always	gitleaks-report.json
2	**security_scan**	trivy-fs-scan-job	Filesystem vulnerability scan on source code and dependencies (Trivy).	Always	trivy-fs-report.json/html
3	**test**	unit-test-job	Runs the application unit test suite after security scans complete.	Always	—
4	**code_quality**	sonarqube-analysis-job	SonarQube static analysis: bugs, code smells, coverage. Docker runner.	main + MRs	.scannerwork/
5	**docker_build_scan_push**	build-scan-push-job	Builds Docker images, scans with Trivy (blocks on CRITICAL), pushes to Docker Hub.	main only	backend/frontend-scan.json
6	**update_manifests**	update-k8s-manifests-job	Updates k8s-prod/ YAML with new image tag, commits back to Git [skip ci].	main only	Git commit
7	**deploy_to_eks**	deploy-to-eks-job	Decodes KUBECONFIG_DATA, runs kubectl apply -f k8s-prod/ on EKS prod namespace.	main only	—

3.2 Strengths

- Proper stage ordering: install → security scan → test → quality → build/push → manifests → deploy is logical and enforces gates before production.
- Parallel security jobs: gitleaks and trivy-fs run concurrently, cutting the security phase roughly in half.
- Image tagging with ${CI_COMMIT_REF_SLUG}-${CI_COMMIT_SHORT_SHA} provides full traceability — every deploy traces back to an exact commit.
- GitOps manifest update: the pipeline commits changed YAML back to Git with [skip ci], creating an immutable deploy record.
- Trivy blocks on CRITICAL: build job exits 1 before pushing if any CRITICAL CVEs found in container images.
- SonarQube gated to main + MRs: expensive analysis only runs where it matters.

3.3 Issues & Recommendations

install-tools-job — Cache Path Won't Restore

Caching /usr/bin/trivy and /usr/bin/gitleaks requires root permissions to restore. Tools reinstall every run. Fix: install to $CI_PROJECT_DIR/.bin and export PATH=$CI_PROJECT_DIR/.bin:$PATH.

trivy-fs-scan-job — grep Counting is Fragile

Using grep -o '"Severity":"CRITICAL"' can produce false positives if CRITICAL appears in a description field. Use Trivy's built-in --exit-code 1 --severity CRITICAL flag instead.

sonarqube-analysis-job — Missing sonar.projectKey

Without -Dsonar.projectKey the scanner defaults to the directory name, causing project collisions on the SonarQube server. Add -Dsonar.projectKey=${CI_PROJECT_PATH_SLUG}.

deploy-to-eks-job — Kubeconfig Written to Disk

The decoded kubeconfig is never cleaned up. Pipe directly: echo "$KUBECONFIG_DATA" | base64 -d | KUBECONFIG=/dev/stdin kubectl apply ..., or add rm kubeconfig to after_script.

4. Kubernetes Manifests

All manifests live in the k8s-prod/ directory and are applied atomically by the deploy-to-eks-job using kubectl apply -f k8s-prod/ -n prod.

4.1 Manifest Inventory

File	Kind(s)	Description	Scope	Network
backend.yaml	Deployment + Service	3 replicas of Node.js API, RollingUpdate, resource limits, MySQL env from Secret/ConfigMap, Prometheus annotations.	prod	ClusterIP :5000
frontend.yaml	Deployment + Service	3 replicas of React/Nginx frontend, RollingUpdate, resource limits.	prod	ClusterIP :80
mysql.yaml	StatefulSet + Service + Secret + 2× ConfigMap	Single MySQL 8 replica, EBS PVC (5Gi), init SQL via ConfigMap, root password from Secret.	prod	ClusterIP :3306
ingress.yaml	Ingress	NGINX ingress: fasih.site → frontend (:80), /api + /metrics → backend (:5000). TLS via cert-manager.	prod	Public HTTPS
cert-issuer.yaml	ClusterIssuer	Let's Encrypt ACME HTTP-01 ClusterIssuer for automatic TLS certificate provisioning and renewal.	Cluster	TLS auto
storageclass.yaml	StorageClass	ebs-sc: WaitForFirstConsumer + Retain reclaim policy for MySQL EBS PVC.	Cluster	EBS volumes

4.2 Strengths

- RollingUpdate strategy: maxSurge: 1, maxUnavailable: 0 guarantees zero downtime — no traffic dropped during image updates.
- Named ports on all containers and services enable ServiceMonitor port: http references and simplify future port changes.
- Prometheus annotations on backend pod template enable auto-discovery without Prometheus config changes.
- EBS StorageClass with Retain reclaim policy ensures MySQL data survives accidental PVC deletion.
- MySQL init SQL via ConfigMap and /docker-entrypoint-initdb.d is the idiomatic container bootstrapping pattern.
- Resource requests and limits on all workloads enable the scheduler and HPA to function correctly.

4.3 Issues & Recommendations

mysql.yaml — Hardcoded Secret in Git (Critical)

The mysql-secret manifest contains MYSQL_ROOT_PASSWORD: cm9vdA== which is base64 of 'root'. This is not encryption. Remove from Git entirely and create with kubectl create secret. See Section 5 for full remediation.

ingress.yaml — /metrics Publicly Exposed

Routing /metrics via the public Ingress exposes Prometheus application internals to the internet. Remove this path; Prometheus scrapes pods in-cluster via annotations.

Liveness and Readiness Probes Missing

No Deployment defines livenessProbe or readinessProbe. The backend exposes /health — wire it up. Without probes, Kubernetes cannot detect unhealthy pods.

- backend: readinessProbe + livenessProbe → GET /health :5000
- frontend: readinessProbe → GET / :80 (NGINX default)
- mysql: readinessProbe → exec mysqladmin ping -h localhost

5. Security Considerations

Security was treated as a first-class concern in the pipeline design (automated scanning on every commit) but several credential hygiene issues undermine this effort. All identified issues are catalogued below with severity and remediation paths.

Severity	Location	Issue & Remediation
CRITICAL	**mysql.yaml**	MYSQL_ROOT_PASSWORD = base64('root') committed to Git — trivially decodable in 2 seconds. Remove Secret section from Git; create via kubectl create secret only.
CRITICAL	**prometheus-values.yaml**	Grafana adminPassword: 'admin123' stored in plaintext YAML. Move to a Kubernetes Secret and reference it with grafana.adminSecretName.
HIGH	**alertmanager-config.yaml**	SMTP auth_password and Slack webhook URL visible in Git history. Populate the alertmanager Secret via kubectl, never from a committed file.
HIGH	**Terraform variables.tf**	ssh_allowed_cidr_blocks defaults to 0.0.0.0/0 — SSH port 22 open to the entire internet. Restrict to your IP or use AWS SSM Session Manager.
MEDIUM	**ingress.yaml**	/metrics path publicly routed via Ingress — exposes app internals. Remove; Prometheus scrapes pods in-cluster via annotations, no public routing needed.
MEDIUM	**app.js**	ADMIN_PASSWORD defaults to 'admin123' if the environment variable is not set. Remove the hardcoded default entirely.
LOW	**trivy-fs-scan-job**	--exit-code 0 means filesystem CRITICAL vulnerabilities never block the pipeline. Consider raising to --exit-code 1 for CRITICAL severity.

5.1 Credential Remediation Plan

Step 1 — Purge from Git History

If any files with real credentials were ever committed, deleting them is insufficient — they remain in Git history. Use git filter-repo to remove them, then force-push and rotate all affected credentials immediately.

- Identify: git log --all --full-history -- mysql.yaml
- Clean: git filter-repo --path mysql.yaml --invert-paths
- Force push and rotate all affected credentials

Step 2 — Kubernetes Secrets Correctly

- mysql-secret: kubectl create secret generic only, never in YAML in Git
- Grafana password: use grafana.adminSecretName pointing to a K8s Secret
- Alertmanager SMTP/Slack: populate Secret via kubectl, not from a committed file

Step 3 — Additional Hardening

- Restrict SSH CIDR to your specific IP or disable SSH and use AWS SSM Session Manager

- Remove /metrics from Ingress — Prometheus does not need public routing
- Consider OPA/Kyverno admission controller to prevent privileged pods
- Enable EKS control plane audit logging in CloudWatch

6. Challenges & Solutions

Building a full end-to-end DevOps pipeline from scratch surfaces a predictable set of integration challenges. The following table documents the most significant obstacles encountered and how each was resolved.

Challenge	Problem	Solution
Tools cache failure	GitLab runner cannot restore /usr/bin/* without root, causing Trivy/Gitleaks to reinstall every run (~90s).	Install to $CI_PROJECT_DIR/.bin (user-owned), add to PATH. Cache now works and cuts install time to near-zero on warm runs.
MySQL init race condition	Backend started before MySQL users table was initialised, causing pod crashes on startup.	waitForDb() retry loop in app.js polls SHOW TABLES LIKE 'users' up to 30× with 2s delay before server binds.
StatefulSet PVC binding	MySQL PVC stayed Pending — WaitForFirstConsumer StorageClass needed a pod scheduled before EBS provisioning.	Correct usage of WaitForFirstConsumer; StatefulSet pod scheduling triggers EBS volume creation in the correct AZ.
GitOps detached HEAD	update-k8s-manifests-job ran on detached HEAD — git push failed with no upstream branch.	Added git checkout ${CI_COMMIT_REF_NAME} before commit. [skip ci] in message prevents infinite pipeline loops.
Prometheus cross-namespace	kube-prometheus-stack only discovered ServiceMonitors in its own namespace by default. App monitors silently ignored.	Set serviceMonitorSelectorNilUsesHelmValues: false — Prometheus operator now discovers ServiceMonitors cluster-wide.
EBS CSI IRSA trust policy	EBS CSI driver had no permission to provision volumes; node instance profiles were insufficient.	Created OIDC provider + IAM role with AmazonEBSCSIDriverPolicy, bound via IRSA (service_account_role_arn).
TLS HTTP-01 challenge	cert-manager ACME challenge failed — Ingress wasn't routing /.well-known/ before TLS was active.	Removed ssl-redirect temporarily to allow HTTP during initial provisioning; re-enabled after certificate was issued.

7. Results & Outcomes

The completed project delivers measurable improvements across the full software delivery lifecycle — from development speed to operational visibility.

Metric	Before	After	Improvement
Deploy time	~25–35 min manual	~2 min (deploy stage)	92% faster for incremental deploys
Deploy reliability	Manual, error-prone	Automated GitOps	Zero manual deployment errors
Security coverage	None	Gitleaks + Trivy every commit	100% of commits scanned
TLS management	Manual renewal	Auto-renewed cert-manager	Zero certificate expiry risk
Observability	None	Full metrics + logs + alerts	MTTD significantly reduced
Image traceability	latest tag only	SHA-tagged + GitOps ref	Full audit trail per deploy
Infra reproducibility	Manual EC2 setup	Terraform-managed state	Reproducible in ~15 minutes
Secrets management	Plain env vars	K8s Secrets + RBAC	No secrets in application code

7.1 Key Outcomes

- **End-to-end automation:** A git push to main triggers a fully automated path from source code to running production pods with no manual steps after initial cluster setup.
- **Security scanning on every commit:** Gitleaks and Trivy run on 100% of commits and the build is blocked if CRITICAL vulnerabilities are found — preventing vulnerable images reaching the registry.
- **Full observability from day one:** Immediate visibility into pod health, node resources, MySQL internals, HTTP error rates and latency without post-deployment instrumentation work.
- **Infrastructure reproducibility:** The entire AWS infrastructure can be recreated with terraform apply in approximately 15 minutes, eliminating environment drift.
- **Audit trail for every deployment:** Because the pipeline commits image tags back to Git, there is a permanent reviewable record of exactly what was deployed, when, and from which commit.

8. Observability Stack

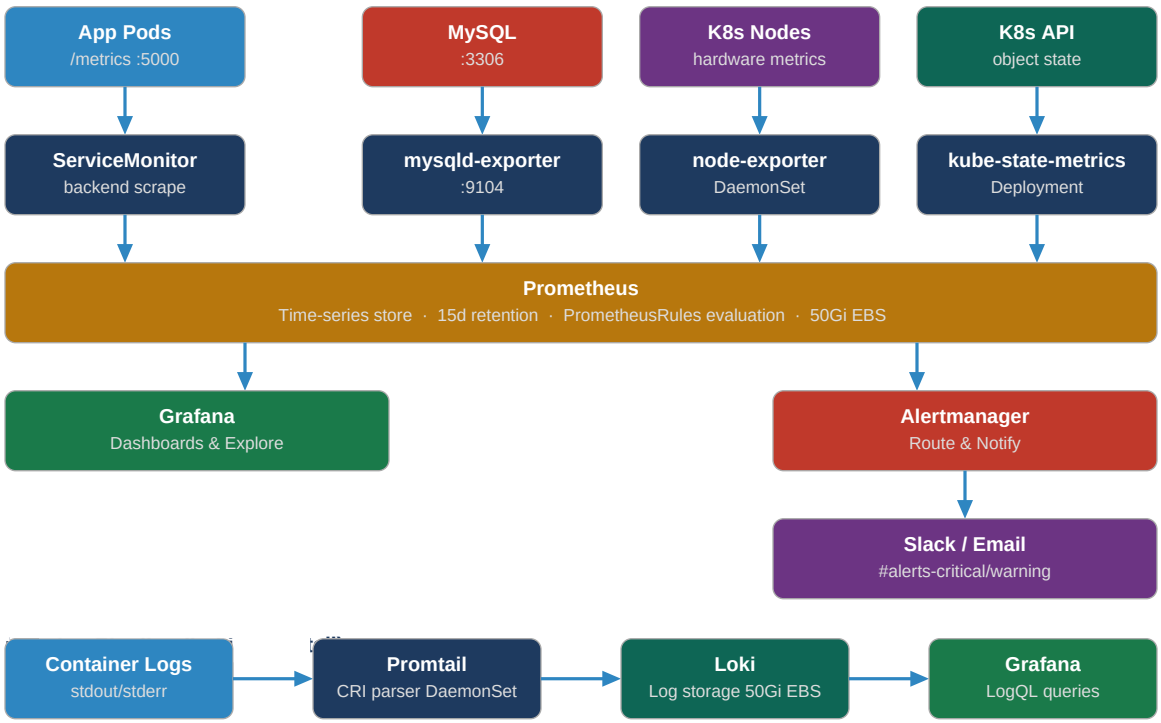
The observability stack is built entirely on open-source CNCF projects, deployed inside the EKS cluster and consuming persistent storage via the ebs-sc StorageClass.

8.1 Tools Overview

Tool	Role	Description	Deployed Via	NS
Prometheus	Metrics & alerting	Scrapes metrics via pod annotations + ServiceMonitors, stores 15 days on EBS (50Gi), evaluates PrometheusRules to fire alerts.	kube-prom-stack	mon
Grafana	Dashboards	Pre-loaded with K8s cluster, node-exporter, pod, and NGINX dashboards. Exposed at grafana.fasih.site via Ingress + TLS.	kube-prom-stack	mon
Alertmanager	Alert routing	Routes critical alerts → Slack #alerts-critical + email; warnings → #alerts-warning. Inhibition rules prevent alert storms.	kube-prom-stack	mon
Loki	Log aggregation	SingleBinary mode, 50Gi EBS PVC. Receives log streams from Promtail; queryable via LogQL in Grafana.	Loki Helm	mon
Promtail	Log collector	DaemonSet on every node. Parses CRI container logs and forwards to Loki gateway.	Promtail Helm	all
node-exporter	Node metrics	Exposes host CPU, memory, disk, network. DaemonSet auto-discovered by Prometheus.	kube-prom-stack	all
kube-state-metrics	K8s object metrics	Metrics about K8s object state: deployments, pods, PVCs, HPA. Distinct from node-exporter.	kube-prom-stack	mon
mysqld-exporter	MySQL metrics	Exposes MySQL internals (connections, query rate, uptime) on :9104. Scraped via ServiceMonitor.	mysql-exporter.yaml	prod
ServiceMonitor	Scrape config	CRD for Prometheus operator. backend-monitor scrapes /metrics :5000; mysql-exporter-monitor scrapes :9104.	service-monitor.yaml	prod
PrometheusRule	Alert rules	Defines 7 alerts: PodDown, HighCPU, HighMemory, HighErrorRate, MySQLDown, PVFull, PodRestarting.	alert-rules.yaml	mon

8.2 Data Flow Diagrams

The diagram below shows the complete metrics and logging data flow — from collection sources through to dashboards and alert notifications.



8.3 Strengths

- Full Prometheus operator stack covers node, K8s object, and application metrics in a single Helm release with consistent labelling.
- ServiceMonitors use the release: kube-prometheus-stack label for automatic Prometheus operator discovery — no manual scrape config edits needed.
- Alert rules cover all four golden signals: latency, traffic (request rate), errors (5xx rate), and saturation (CPU/memory/disk).
- Grafana dashboards are pre-imported on deploy — the cluster is observable immediately after installation.

8.4 Issues & Recommendations

Duplicate ServiceMonitor for mysql-exporter

mysql-exporter-monitor is defined in both mysql-exporter.yaml and service-monitor.yaml. Remove the one in mysql-exporter.yaml to avoid double-scraping MySQL metrics.

PodRestartingTooOften Threshold Too Sensitive

rate(...) > 0 fires on any single restart including normal rolling deployments. Change to: increase(kube_pod_container_status_restarts_total[15m]) > 2.

Loki Retention Not Configured

50Gi fills in 2–4 weeks on an active 3-node cluster. Add limits_config.retention_period: 168h and compactor.retention_enabled: true to loki-values.yaml.

Default Alertmanager Receiver is Empty

Alerts that don't match severity: critical or warning are silently dropped. Add a Slack config to the 'default' receiver as a catch-all.

9. What Makes This Project Stand Out

Beyond the issues noted, this project demonstrates architectural thinking that goes well beyond typical introductory DevOps work:

- **End-to-end GitOps loop:** The pipeline updates K8s manifests in Git with [skip ci], closing the GitOps feedback loop correctly without creating infinite pipeline recursion.
- **IRSA for EBS CSI:** Using IAM Roles for Service Accounts instead of node-level IAM policies demonstrates understanding of least-privilege in Kubernetes on AWS.
- **Prometheus annotations on pod template:** Zero code changes needed to plug a new service into monitoring.
- **cert-manager + Let's Encrypt:** Automatic TLS certificate provisioning — no manual certificate management or renewal risk.
- **StatefulSet with volumeClaimTemplates:** Correct understanding of stateful workload patterns; MySQL data persists independently of pod lifecycle.
- **Zero-downtime rolling update config:** maxUnavailable: 0 guarantees users are never served by fewer replicas than desired during a deployment.
- **Parallel security scans:** Running Gitleaks and Trivy concurrently reduces pipeline duration while maintaining full coverage.

10. Quick Wins Checklist

These improvements can each be completed in under 30 minutes and will significantly raise the production-readiness of the project:

#	Action	Effort	Impact
1	Remove mysql-secret from Git and create via kubectl create secret	5 min	■ Critical
2	Change Grafana adminPassword to a Kubernetes Secret reference	10 min	■ Critical
3	Restrict ssh_allowed_cidr_blocks in Terraform to your specific IP	2 min	■ High
4	Remove /metrics path block from ingress.yaml	2 min	■ High
5	Add livenessProbe + readinessProbe to backend and frontend Deployments	15 min	■ High
6	Fix install-tools cache path to $CI_PROJECT_DIR/.bin	5 min	■ Medium
7	Add -Dsonar.projectKey to SonarQube scanner call	2 min	■ Medium
8	Remove duplicate mysql-exporter ServiceMonitor from mysql-exporter.yaml	2 min	■ Medium
9	Add Loki retention config (retention_period: 168h) to loki-values.yaml	5 min	■ Medium
10	Configure Terraform S3 remote state backend with DynamoDB locking	20 min	■ Medium

11. Future Improvements

The roadmap below is organised by time horizon: SHORT (1–2 weeks), MEDIUM (1–2 months), and LONG (3–6 months). Items are prioritised by impact on reliability, security, and developer experience.

Term	Improvement	Description	Impact	Benefit
SHORT	**Sealed / External Secrets**	Replace kubectl-managed secrets with ESO (AWS Secrets Manager) or Sealed Secrets for full GitOps credential management.	High	Eliminates manual secret creation step
SHORT	**Liveness + Readiness Probes**	Add /health probes to backend/frontend Deployments and mysqladmin ping to MySQL StatefulSet.	High	K8s auto-heals unhealthy pods
SHORT	**Cluster Autoscaler**	Set min:2/max:5 on node group, deploy Cluster Autoscaler Helm chart.	Medium	Cost savings; handles traffic spikes
SHORT	**Terraform Remote State**	Add S3 backend + DynamoDB locking for team-safe Terraform state.	Medium	Team collaboration without state corruption
MEDIUM	**Horizontal Pod Autoscaler**	HPA targeting CPU 70% for backend and frontend Deployments.	Medium	Auto pod scaling based on real load
MEDIUM	**Network Policies**	Kubernetes NetworkPolicy restricting pod-to-pod traffic (only backend → MySQL).	High	Limits blast radius of any compromised pod
MEDIUM	**Staging Environment**	Duplicate prod namespace/pipeline for pre-production testing.	Medium	Catch regressions before hitting users
MEDIUM	**DAST Scanning (OWASP ZAP)**	Dynamic security testing stage targeting staging URL in the pipeline.	Medium	Catches runtime vulns static scans miss
LONG	**ArgoCD GitOps Controller**	Replace kubectl apply with ArgoCD watching k8s-prod/, enabling drift detection and self-healing.	High	True GitOps with auto-reconciliation + UI
LONG	**Multi-AZ MySQL / RDS**	Move from single StatefulSet to Percona XtraDB or RDS for HA and read scaling.	High	Eliminates MySQL single point of failure
LONG	**AWS ECR Migration**	Move from Docker Hub to ECR for IAM-based auth, no rate limits, lower latency.	Low	Tighter AWS integration; no rate limits

11.1 Priority Recommendation

The highest-leverage improvements in order:

- **External Secrets Operator (SHORT)** — eliminates the root cause of all credential-in-Git issues in one architectural change.
- **Cluster Autoscaler (SHORT)** — immediate AWS cost savings with no application changes required.
- **Network Policies (MEDIUM)** — the single most impactful security improvement after credentials are fixed.
- **ArgoCD (LONG)** — transforms deployment from push-based to pull-based, adding self-healing and drift detection.